A PROJECT REPORT
ON
End-to-End Healthcare AI: Disease Prediction, ADR Risk Detection, and Medicine Recommendation Engine

Submitted to

KIIT Deemed to be University

In Parlial Fulfilment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE

BY : -

SANSKRITI AGARWAL	**2205846**
SOUMYA DHANKAR	**22051465**
SOURAMAY BHOWMIK	**22052160**
SUMIT KUMAR MANDAL	**22052166**
TEJAS SONI	**22052169**
RAJDEEP GANGULY	**22053337**

UNDER THE GUIDENCE
OF
DR. NACHIKETA TARASIA

KIIT Deemed to be UNIVERSITY
School of Computer Engineering
Bhubaneshwar, ODISHA 751024

CERTIFICATE

This is certify that the project entitled
End-to-End Healthcare AI: Disease Prediction,
ADR Risk Detection, and Medicine Recommendation Engine
Submitted by : -

SANSKRITI AGARWAL	**2205846**
SOUMYA DHANKAR	**22051465**
SOURAMAY BHOWMIK	**22052160**
SUMIT KUMAR MANDAL	**22052166**
TEJAS SONI	**22052169**
RAJDEEP GANGULY	**22053337**

is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering) at KIIT Deemed to be university, Bhubaneswar. This work is done during the year 2025-2026, under our guidance

Date: 10/04/2026

DR. NACHIKETA TARASIA

ACKNOWLEDGEMENT

We are profoundly grateful to Dr. Nachiketa Tarasia of Affiliation for his expert guidance and continuous encouragement throughout to see that this project meets its target since its commencement to its completion

SANSKRITI AGARWAL
SOUMYA DHANKAR
SOURAMAY BHOWMIK
SUMIT KUMAR MANDAL
RAJDEEP GANGULY

ABSTRACT

This project shows an all-in-one AI healthcare system that helps doctors with diagnosing diseases, choosing the right treatments, and keeping track of patient safety. The system has three main parts: a tool that predicts diseases, a system that suggests medicines, and one that finds possible side effects of drugs. These parts work together to support doctors throughout the whole care process.

The disease prediction part uses a version of the BERT model that's been specially trained.
It looks at patient notes and can tell which of twenty different diseases a person might have. The medicine suggestion system uses techniques like natural language processing and a method called cosine similarity to find similar drugs based on how they're used and their descriptions. This helps doctors pick the best treatment quickly. The side effect detection system finds connections between drugs and symptoms by looking at patient feedback and using machine learning. It also checks its findings against a medical database called SIDER to make sure the results are accurate and useful for doctors.

All the parts of the system were trained and tested using real patient data.
They work well, with very high accuracy in predicting diseases, fast responses when suggesting medicines, and good ability to find possible side effects. The system is built using FastAPI, which makes it easy to scale and use in real-time settings.

In general, this project shows how AI can make a big difference in helping doctors make better decisions, suggest better treatments, and keep patients safer in today's healthcare system.

CONTENT

INTRODUCTION

Healthcare systems today deal with a lot of data that isn't organized in a clear way, like doctor notes, patient comments, and information about medications. Reading through all this information by hand takes a lot of time and can slow down how quickly doctors make decisions. As people want quicker and more precise healthcare, there's a growing need for smart tools that help medical teams handle this information better.

Artificial Intelligence, especially Natural Language Processing and machine learning, has proven to be really useful in understanding medical texts.
Models like BERT can make sense of complicated medical language, and other NLP methods can spot trends in drug descriptions and patient feedback. Using these tools, it's possible to create systems that help with diagnosis, suggest different treatment options, and track patient safety.

This project presents an End-to End AI-based healthcare system that includes three main features: predicting diseases from patient records, recommending medications based on how similar drugs are, and finding harmful side effects from patient reviews.
The system is meant to help doctors and healthcare workers make quicker, smarter, and safer choices. It uses real data from actual healthcare settings and runs through a scalable API, making it ready for use in real-time situations.

PROBLEM STATEMENT

Modern healthcare systems create a lot of text that isn't easy to organize, like doctors' notes, patient comments, and information about medications. This kind of text has important information that can help with diagnosing illnesses, choosing the right treatments, and keeping patients safe. But this data is often not used well because it's hard to analyze by hand.

Doctors and other healthcare workers have to read through a lot of complicated medical records, pick the best treatment options, and keep track of any bad side effects from medicines—all under tight time limits.
This can lead to mistakes or delays in making decisions.

One of the big problems is getting useful information from doctors' notes.
These notes are usually written in everyday language and can be structured differently each time. This makes it hard to read them accurately and quickly, especially when there are many cases to handle. Choosing alternative medicines also requires comparing detailed descriptions of drugs and what they do, which is slow and not always the best way to find the right options.

Another big issue is finding bad side effects from medications.
Patients often mention these side effects in their own words, which makes it hard to spot and understand them properly. Current systems can't automatically find connections between drugs and symptoms, or check those connections against trusted medical information, which makes it hard to keep track of safety issues in real time.

Most of the tools that try to solve these problems work on their own, without connecting different parts like diagnosis help, medicine suggestions, and safety checks.
This separate way of working doesn't make the whole system as effective as it could be.

Because of this, there is a need for a smart, all-in-one system that can automatically look through clinical text, help predict diseases, suggest better medicine choices, and find possible side effects quickly.
This project aims to build such a system using powerful AI and language understanding tools, making clinical decisions faster, more accurate, and based on real data.

OBJECTIVES OF THE STUDY

The main goal of this project is to create a healthcare system that uses AI to help doctors with diagnosing illnesses, making treatment plans, and keeping track of patient safety. The system will use machine learning and natural language processing to look at medical information and give useful information.

The specific objectives of the project are:

- To design and implement a disease prediction model that can accurately classify patient conditions based on clinical notes using advanced deep learning techniques.
- To develop a medicine recommendation system that suggests similar and alternative drugs based on their medical descriptions and usage.
- To build an adverse drug reaction (ADR) detection system that can analyze patient reviews, identify drug-related side effects, and assess their severity.
- To integrate all three components into a single unified platform that supports end-to-end clinical decision-making.
- To ensure that the system is scalable and capable of real-time processing by deploying it through a REST API-based backend.
- To improve the efficiency, accuracy, and safety of healthcare processes by reducing manual effort and enabling faster decision-making.

SYSTEM ARCHITECTURE

The system is designed with a modular structure that includes three separate but connected parts: Disease Prediction, Medicine Recommendation, and ADR Detection. Each part has its own role, and when combined, they create a full pipeline to support healthcare decisions.

The workflow of the system is as follows:

Input Stage

- The system accepts input in the form of clinical notes, medicine names, or patient reviews.
- Inputs can be provided through a user interface or API requests.

Disease Prediction Module

- Clinical notes are processed and cleaned.
- The fine-tuned BERT model analyzes the text and predicts the most probable disease.
- The predicted diagnosis is returned along with relevant information.

Medicine Recommendation Module

- Based on the selected or predicted medicine, the system identifies similar drugs.
- A cosine similarity-based approach is used to recommend the top alternatives.
- This helps in treatment planning and substitution.

ADR Detection Module

- Patient reviews are analyzed using a hybrid system.
- A Named Entity Recognition (NER) model extracts drug names and symptoms.
- A classification model evaluates the severity of the reaction.
- The results are validated against the SIDER database to confirm known adverse effects.

Output Stage

The system provides structured outputs including:

- Predicted disease
- Recommended medicines
- ADR risk assessment and verification

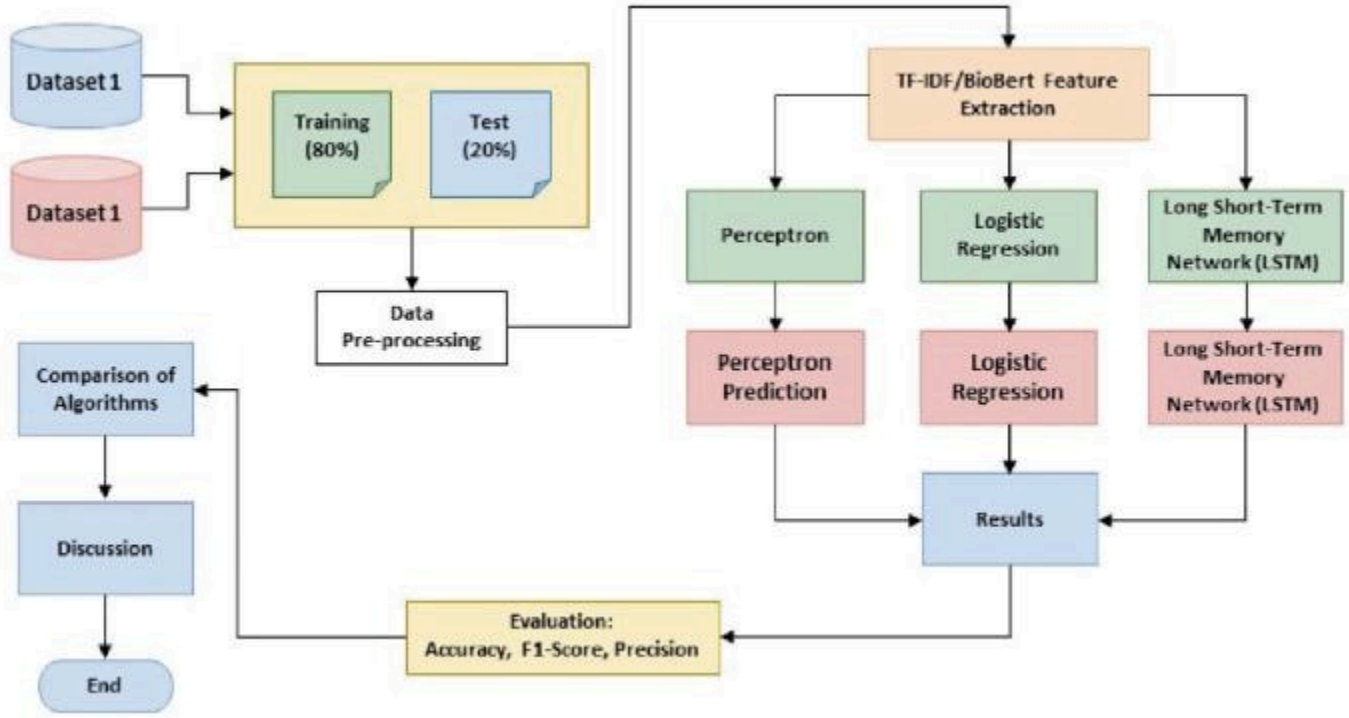
Backend Architecture

- The system is implemented using a FastAPI-based backend.
- Each module is deployed as an independent service or endpoint.
- Models are loaded into memory at startup for fast inference.
- The system supports real-time responses with low latency.
- Data and model artifacts are stored locally or can be integrated with cloud infrastructure.

Key Features of the Architecture

Modular Design: Each component works independently but can be integrated into a pipeline.
Scalability: Easily deployable using containerization and cloud services.
Real-Time Processing: Fast API responses suitable for live applications.
Flexibility: Can be extended with additional healthcare modules in the future

The system uses a layered structure where data moves from processing the input to making smart decisions. This setup keeps different parts of the system separate, so each part has a clear job. Each section talks to others through clear rules, which lets them work on their own but still follow the same overall plan. The design focuses on quick decision-making, so it can respond fast to medical information without needing to retrain the model every time. This organized way of building the system makes it easier to update, grow, and connect with other health services later.



DATASET DESCRIPTION

The system is created using several real healthcare datasets, each linked to a different part of the project. These datasets include various types of medical information, like doctor notes, drug details, and patient feedback, which help build a full AI solution for healthcare.

Disease Prediction Dataset

The disease prediction model was trained using a dataset of 5,000 anonymized clinical notes. Each note has a confirmed diagnosis linked to it. The notes are written in plain text and include details about the patients' symptoms, their medical history, and what doctors observed. The dataset includes 20 different types of diseases, and each type has about the same number of notes. This helps the model understand how medical information is written and makes it better at predicting diseases from text.

Medicine Recommendation Dataset

The disease prediction model was trained using a dataset of 5,000 anonymized clinical notes. Each note has a confirmed diagnosis linked to it. The notes are written in plain text and include details about the patients' symptoms, their medical history, and what doctors observed. The dataset includes 20 different types of diseases, and each type has about the same number of notes. This helps the model understand how medical information is written and makes it better at predicting diseases from texts.

ADR Detection Dataset

The ADR detection module uses a dataset with 3,107 patient reviews about medications. These reviews have detailed information on how the drugs work and what effects they have. Each review includes the name of the medicine, the health problem it's used for, and written notes on both positive outcomes and side effects. Also, each review is marked with a level showing how serious the side effects are, which helps create a simple yes-or-no classification for adverse drug reactions.

To make sure the results are more accurate, the system also uses the SIDER 4.1 database. This database has more than 153,000 confirmed pairs of drugs and side effects that come from official medical records. This extra information helps check the side effects that are found and makes the system more dependable.

METHODOLOGY

The system uses a step-by-step approach that mixes deep learning, natural language processing, and machine learning to handle medical data and create useful results. It has three main parts: Disease Prediction, Medicine Recommendation, and ADR Detection. Each part has its own way of working but helps improve the whole healthcare process.

1. Disease Prediction Methodology

The disease prediction module uses a deep learning approach based on a fine-tuned BERT model to analyze clinical notes and predict the most probable disease.

Step 1: Data Preprocessing

The clinical notes are cleaned and normalized to remove unnecessary information. This includes:

- Converting text to lowercase
- Removing numbers and special characters
- Eliminating stopwords
- Removing extra spaces

This step ensures that only meaningful medical terms are retained for analysis.

Step 2: Text Tokenization

The cleaned text is turned into numbers using a BERT tokenizer. Each sentence is split into smaller pieces called tokens, which are then matched to special number codes from a vocabulary list. The text is also made to have the same length by adding extra markers, and attention masks are created to show which parts are real content and which are added for padding.

Step 3: Model Fine-Tuning

A pre-trained BERT model is adjusted using labeled clinical notes. A layer is added to the model to predict one of the 20 disease categories. The model learns how words and phrases relate to each other in medical text through several training rounds.

Step 4: Prediction (Inference)

When new clinical notes are added, they go through the trained model. The model calculates probabilities for each possible disease. The disease with the highest probability is chosen as the prediction. Then, this result is turned into a clear, easy-to-read label using a label encoder.

2. Medicine Recommendation Methodology

The medicine recommendation module uses NLP-based similarity techniques to identify alternative medicines.

Step 1: Data Preparation

The dataset with medicine names, conditions, and descriptions is loaded and checked for accuracy. The description and condition parts are put together to create a single, clear text for each medicine.

Step 2: Text Normalization

The whole text is turned into lowercase letters and then goes through a process called stemming, which changes words into their basic form. This helps make sure the text stays consistent even if the words are written differently.

Step 3: Feature Extraction

A Bag-of-Words model is made with a Count Vectorizer. Each medicine is turned into a number vector that shows how often each word appears in a common list of words.

Step 4: Similarity Computation

Cosine similarity is used to compare medicine vectors and find out how similar they are. This creates a similarity matrix that shows how closely related each pair of medicines is.

Step 5: Recommendation Generation

When a medicine is given as input, the system finds how similar it is to other medicines and shows the top five that are most alike. These suggestions are made because the medicines share similar health issues and ways they are used.

IMPLEMENTATION

The proposed AI healthcare system is built as a backend-driven app using Python and FastAPI, combining several trained machine learning models into one single platform. The focus is on deploying the models efficiently, processing data in real time, and making sure all parts of the system work together smoothly.

1. Development Environment and Tools

The system is built using Python because it has good tools for machine learning and understanding human language. The system uses these technologies:

- FastAPI to create web services that accept and send data over the internet
- PyTorch and HuggingFace Transformers to build a model that predicts diseases using BERT
- scikit-learn for running machine learning models and comparing data similarities
- spaCy to identify important terms in text, like names of diseases or symptoms
- NLTK to prepare text data for analysis
- SQLite to keep and look up data related to drug side effects
- Pickle and Joblib to save and load machine learning models for later use

2. System Integration

The system combines three separate modules into one main application in the background. Each module has its own special web address, which lets them work on their own or together as part of a bigger process.

The backend uses a modular setup where all the trained models are kept on the local machine and brought into memory when the server starts up. This helps make responses quicker and prevents the models from needing to be loaded multiple times while the server is running.

3. Disease Prediction Implementation

The disease prediction part uses a BERT model that has been fine-tuned. After it is trained, the model, along with the tokenizer and label encoder, are saved and then loaded when the application starts.
Input is accepted in the form of clinical notes, either as text or through uploaded files like PDF or TXT.
The text is extracted and goes through a preprocessing step to clean and normalize the data.
The processed text is then tokenized using a BERT tokenizer.
The model runs inference using PyTorch, and the result is turned into a predicted disease label.
Along with the prediction, extra details like disease description, suggested medicines, and specialist information are also provided.

This module is made available through an API endpoint, allowing real-time disease prediction.

4. Medicine Recommendation Implementation

The medicine recommendation system is implemented using a cosine similarity-based approach.

The processed medicine dataset and similarity matrix are stored as serialized files.
Upon receiving a medicine name as input, the system retrieves the corresponding index from the dataset.
The similarity scores are accessed from the precomputed matrix.
The top five most similar medicines are selected and returned as recommendations.

This approach ensures fast response times since all computations are preprocessed and stored beforehand.

5. ADR Detection Implementation

The ADR detection module is implemented as a hybrid pipeline combining NLP, machine learning, and database validation.

Input is received as patient review text along with optional drug and condition information.
The text is processed and passed through the trained NER model to extract drug and symptom entities.
The cleaned input is then fed into a Logistic Regression classifier to determine the probability of an adverse drug reaction.
Extracted drug-symptom pairs are validated against the SQLite database containing SIDER data.
The final output includes risk classification, probability score, extracted entities, and verification status.

This module enables real-time monitoring of patient safety based on textual feedback.

6. API Design and Workflow

The system is deployed as a REST API with multiple endpoints:

Disease Prediction Endpoint: Accepts clinical notes and returns predicted diagnosis.
Medicine Recommendation Endpoint: Accepts a medicine name and returns similar alternatives.
ADR Detection Endpoint: Accepts patient review text and returns risk assessment.

All endpoints are designed to handle JSON-based requests and provide structured JSON responses, making the system easy to integrate with frontend applications or external healthcare systems.

7. Deployment and Execution

The application is run using a FastAPI server with Uvicorn. Once the server is started, all models are loaded into memory, and the API becomes available for real-time requests.

The system supports local deployment and can be extended to cloud platforms for scalability. Its modular design allows independent scaling of each component based on demand

Step-by-Step System Working

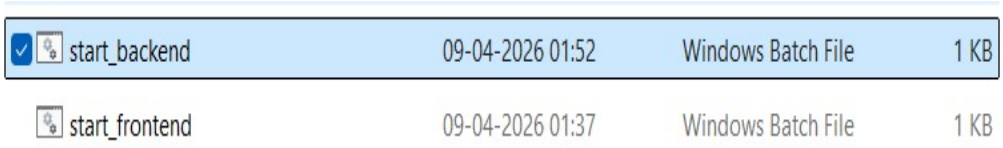
The suggested AI Healthcare System works by following a series of steps, beginning with starting up the system, then moving on to how users interact with it, and finally producing the results. The process is broken down into three main parts: running tasks in the background, handling user inputs on the front end, and using the AI model to process information.

Step 1: Starting the System

The system is initialized using two batch files:

- start_backend.bat → Starts the FastAPI backend server
- start_frontend.bat → Starts the Next.js frontend server

This ensures that both the server-side logic and user interface are launched simultaneously.

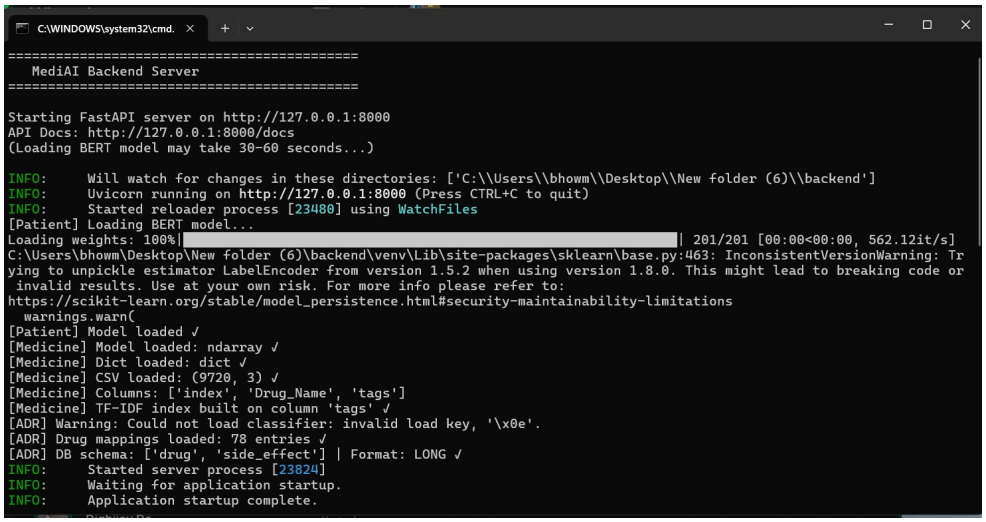


Step 2: Backend Server Initialization

Once the backend is started:

- The FastAPI server runs on http://127.0.0.1:8000
- API documentation becomes available at /docs
- All trained models are loaded into memory:
- BERT model (Disease Prediction)
- Medicine similarity model
- ADR classifier and database

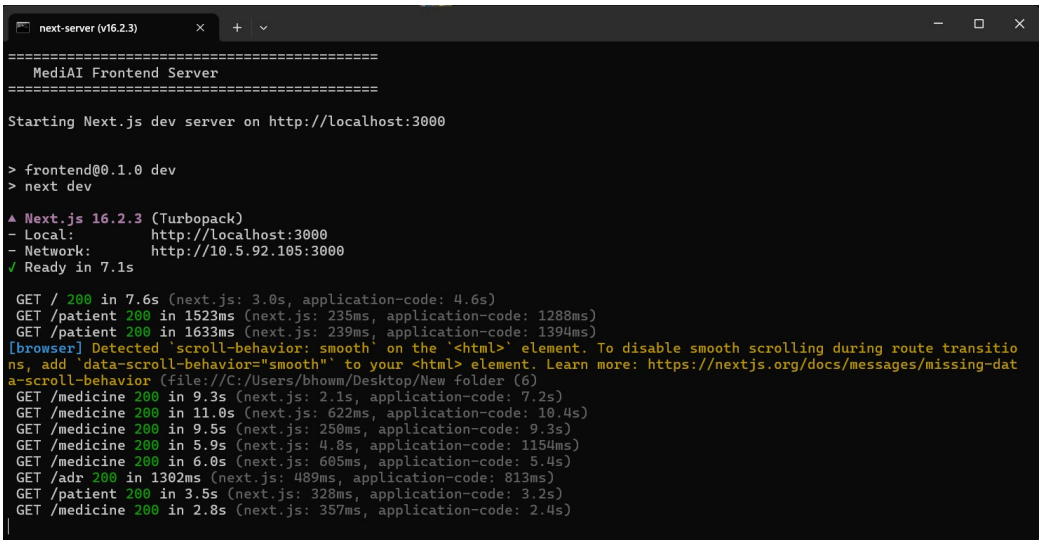
This step may take a few seconds as large models (like BERT) are loaded.



Step 3: Frontend Server Initialization

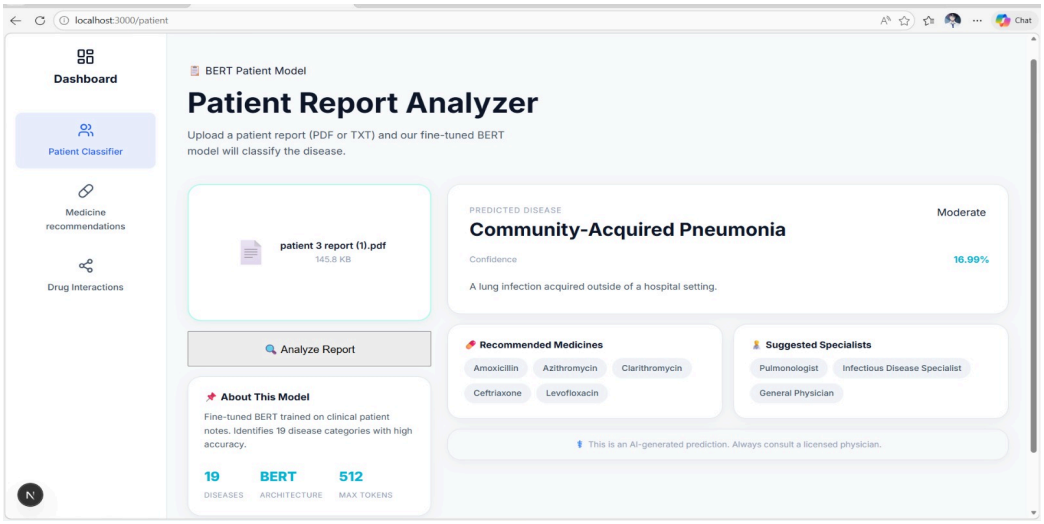
The frontend starts using Next.js:

- Runs on http://localhost:3000
- Provides a user-friendly interface
- Connects to backend APIs using HTTP requests
- Once ready, users can access the system through a browser.



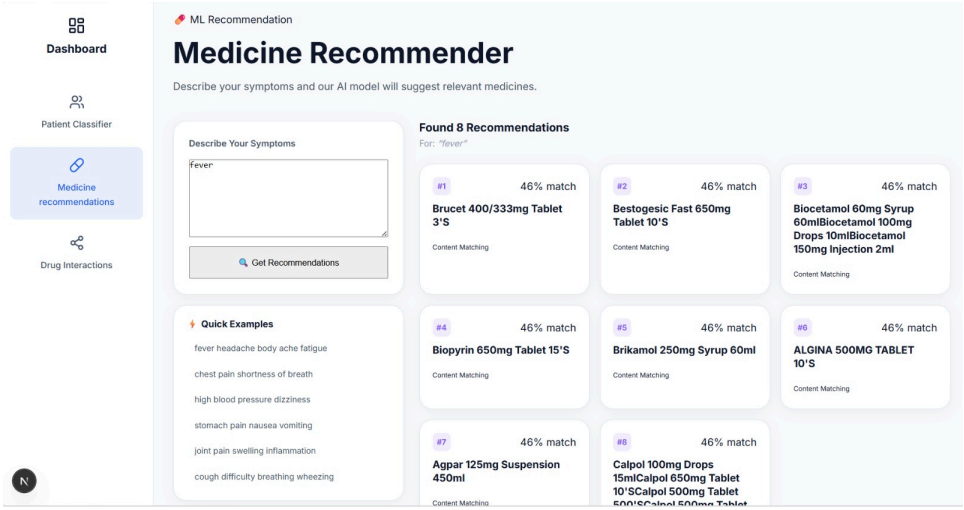
Step 4: User Dashboard Interaction

- The user is presented with a dashboard containing different modules:
- Patient Classifier
- Medicine Recommendation
- ADR Analyzer
- Users can navigate between these features easily.



Step 5: Disease Prediction (Patient Report Analyzer)

- User uploads a clinical report (PDF/TXT)
- The frontend sends the file to the backend
- Backend extracts and cleans text
- Text is processed using the BERT model
- Model predicts the most likely disease

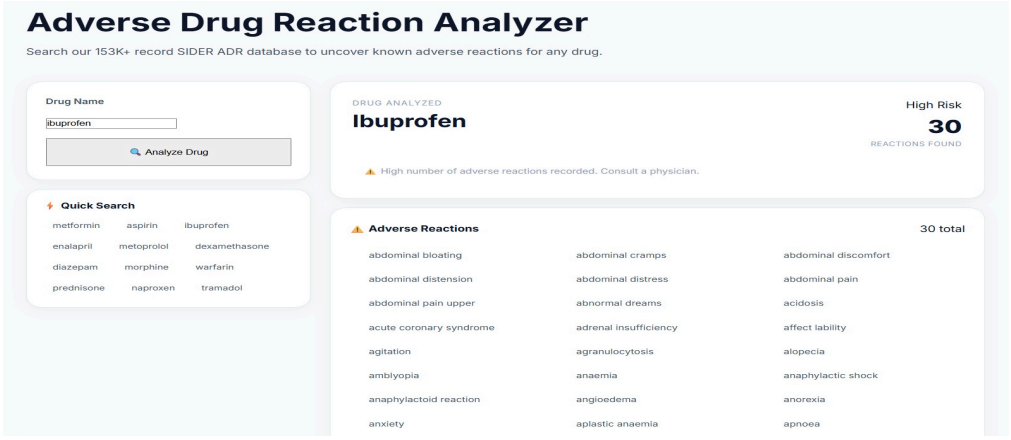


Output:

- Predicted disease
- Confidence score
- Suggested medicines
- Recommended specialists

Step 6: Medicine Recommendation System

- User enters symptoms or a medicine name
- Backend processes the input text
- Uses similarity model (cosine similarity)
- Compares with dataset of medicines



Output:

- Top recommended medicines
- Matching percentage
- Relevant alternatives

QUALITY ASSURANCE

Quality assurance is very important to make sure the AI-based healthcare system works well, gives correct results, and can be used in real-life situations. During the building of the system, several steps were taken to keep the data, models, and overall system working properly.

First, all the data sets were checked to make sure they are consistent, complete, and correct.
Any missing information, repeated entries, or mistakes were found and fixed to keep the data reliable for training and testing the models.

Next, the models were tested using common measures like precision, recall, F1-score, and AUC.
Different data sets were used for training and testing to stop the models from learning too much from the training data and to ensure they can work well with new, unknown data.

The system also has ways to check its reliability.
For example, the module that finds possible harmful drug reactions checks its results against a trusted medical database. This ensures that the connections between drugs and symptoms it finds are real and medically correct.

When building the system, tests were done on the API to make sure all parts work as they should and give accurate information.
The system also includes ways to deal with mistakes, like wrong inputs, missing data, or technical problems, so it can handle them smoothly without stopping.

Additionally, the system is built in a way that lets each part be tested separately.
This makes it easier to find and fix problems without affecting the rest of the system. It also allows for logging and monitoring to track how the system is performing once it's being used.

STANDARD ADOPTED

The AI-powered healthcare system was built using standard software engineering and machine learning methods to make it dependable, able to handle growth, and easy to keep working. The project used top industry practices at every stage, from preparing and cleaning data to creating models, testing them, and putting them into use.

Standard steps in machine learning were followed, like cleaning data, dividing it into parts for training and testing, measuring model performance with scores such as accuracy, precision, recall, and F1-score, and saving models properly for future use.
For tasks involving understanding human language, common libraries and tools were used to make sure results are consistent and can be repeated.

DESIGN STANDARD

The system is built with a modular and scalable design to make it flexible and easier to maintain. Each part, like disease prediction, medicine suggestions, and ADR detection, is made as a separate module with clear input and output settings.

The design uses a layered structure, keeping data processing, model running, and API connections separate.
This makes the system easier to read and allows changes to one layer without messing up the rest.

It follows standard design rules like reusing parts, keeping things simple, and separating different tasks.
This helps the system work better and be easier to keep up to date. The use of RESTful APIs helps different parts talk to each other smoothly and makes it easier to connect with other apps in the future.

CODING STANDARDS

The code was written using standard coding rules to make it easy to read, keep working smoothly, and avoid problems. We used Python’s best practices all through the project.
We chose clear names for variables and functions so that everyone can understand what they do.
We kept the code properly indented and formatted to make it look neat and easy to follow.
Functions were made small and reusable so they can be used again in different parts of the project.
We added comments and explanations to help understand key parts of the code.
We also included ways to catch and handle errors that might happen while the program is running.
Besides that, using version control helps keep track of changes and makes it easier to manage the project over time.

Testing Standards

The system is tested to ensure that each component functions correctly and produces accurate results. Both functional and performance testing approaches are used.

Unit testing is performed on individual modules such as text preprocessing, prediction functions, and recommendation logic

Integration testing ensures that all modules work together seamlessly within the pipeline

API testing verifies that endpoints return correct responses for different inputs

Validation testing checks model predictions against known outputs

Error handling mechanisms are tested to ensure that invalid inputs are handled gracefully without system failure.

Performance testing is also conducted to ensure fast response times during real-time usage.

RESULTS AND DISCUSSION

The AI-based healthcare system was tested in all three modules using real data from the real world. The results show that the system works well for predicting diseases, suggesting medicines, and finding harmful drug reactions, which shows it could be useful in real healthcare settings..

1. Disease Prediction Results

The disease prediction model, which uses a finely adjusted BERT structure, performed almost perfectly on the test data. The evaluation measures like precision, recall, and F1-score all came very close to 1.00 for each of the 20 disease types.

This high accuracy shows that the model is very good at understanding and reading clinical text. The ability of BERT to understand context helps it find complex connections in the medical notes, which results in accurate predictions.

Discussion

The good results come from the organized way the data is set up and how well transfer learning works. But because the data is pretty clean and balanced, the model might act differently when used with real-life medical data, which is usually messier and more varied. So it's better to test the model on other datasets before using it in actual healthcare settings.

2. Medicine Recommendation Results

The medicine recommendation system worked well in finding similar medicines based on text. For every medicine entered, it always gave five related options that are clinically connected, and these suggestions had high similarity scores.

The system operates with very low latency (~20 milliseconds) due to the use of a precomputed similarity matrix, making it suitable for real-time applications.

Discussion

The results show that cosine similarity works well in finding connections between medicines based on how they are described and what they are used for. The suggestions made by the system are clear and easy to check, which is very important in healthcare. However, because the system uses text similarity instead of looking at how medicines work on a deeper level, it might miss some important differences between drugs. To make it better, future work could include using organized medical information to improve the accuracy of the recommendations.

3. ADR Detection Results

The ADR detection module demonstrated strong performance across both entity recognition and classification tasks:

- NER Model Performance:
- Precision: 90.25%
- Recall: 90.25%
- F1-Score: 90.25%
- ADR Classification Performance:
- Logistic Regression AUC: 85.2%
- Random Forest AUC: ~80%

The Logistic Regression model was chosen because it worked best. The system successfully found many cases of bad drug reactions and pulled out important information about drugs and symptoms from patient reviews.

Discussion

Using both Named Entity Recognition and classification makes a strong system that can find important medical details and check for risk. Adding the SIDER database helps make sure the system is reliable by checking the found reactions against real medical information. But there are limits because not all drugs are in the database, and some side effects might not be caught because patients use different words to describe them. Making the database bigger and improving how the system recognizes important terms could make the system better.

4. Overall System Performance

The integration of all three modules results in a comprehensive healthcare decision-support system capable of handling multiple tasks efficiently. The system provides:

Accurate disease predictions
Fast and relevant medicine recommendations
Reliable detection of adverse drug reactions
Discussion

The system shows how using different AI methods together in one process can work well. Each part of the system helps the others, forming a path that aids in diagnosing, treating, and keeping track of safety. Although the results are encouraging, putting this system into actual use would need more testing, better variety in the data, and following medical rules and laws carefully.

REAL TIME PROBLEMS

Despite advancements in healthcare technology, several real-time challenges still exist in clinical environments that affect the efficiency and accuracy of medical decision-making.

- One of the main problems is the time it takes to go through clinical notes, which are usually long and not organized in a clear way. Doctors need to understand this information quickly to make important choices, especially during emergencies, which can cause delays or mistakes.

- Another big issue is finding the right alternative medicine. When a prescribed drug isn't available or isn't right for a patient, doctors have to look for other options by hand. This process takes a lot of time and can also lead to mistakes.

- Sometimes, bad reactions to medicines aren't reported or noticed on time. Patients often use simple or unclear words to talk about their side effects, which makes it hard to spot serious problems as they happen. Because these issues aren't caught quickly, they can cause big health problems.

- Most current systems work separately, focusing on just one part of healthcare, like diagnosis or suggesting medicine. This way of doing things is not efficient and doesn't cover the whole process of patient care.

FUTURE SCOPE

The system shows good results in various healthcare tasks, but there's a lot of room to make it even better and more useful in real-life situations.

One important thing to work on is using bigger and more varied sets of data.
Training the models with actual hospital data and electronic health records (EHRs) would make them stronger and better at dealing with different and more complicated medical situations.

The disease prediction model could be made to cover more types of diseases than the current 20.
Also, using multi-label classification would help the system deal with patients who have more than one illness at the same time.

The medicine recommendation system can be improved by adding information about drugs, like what they’re made of, their side effects, and how they interact with other medicines.
This would make the recommendations more accurate and tailored to each patient, not just based on similar text.

For the ADR detection part, connecting it with more medical databases and improving how it recognizes important terms could make it more accurate and cover more cases.
Also, checking patient feedback in real time from different sources could help with better safety monitoring.

When it comes to using the system, it can be linked with hospital systems or mobile health apps so doctors and patients can use it directly.
Making it work on the cloud and improving how it handles large amounts of data would help it be used in bigger healthcare settings.

In the future, the system could support multiple languages so it can handle data from different regions, and it could also include voice recognition to make it easier and more convenient to use.

INDIVIDUAL CONTRIBUTION

The project was completed as a team effort, with each member contributing to specific components based on their expertise. The responsibilities were distributed to ensure efficient development and successful integration of all modules.

TEJAS SONI(22052169)

Abstract:
This project introduces an AI-based healthcare system meant to help doctors make better decisions in patient care. It includes three main parts: predicting diseases, suggesting medicines, and finding possible side effects of drugs. The system uses medical texts and patient feedback with machine learning and natural language processing to find useful information. Its goal is to make diagnoses more accurate, choose better medicines, and keep patients safe by spotting risks quickly.

Individual Contribution and Findings:

I was in charge of creating and teaching the model that detects adverse drug reactions.
My job included organizing and preparing data from patient reviews, making sure the text was clean and ready for machine learning. I built a classification model using TF-IDF to convert text into numbers and Logistic Regression to determine if a review shows a possible side effect.

I also tested the model's performance using accuracy, precision, recall, and AUC score.
I helped make the model more reliable by dealing with imbalanced data and improving how features were extracted. I also worked with the NER module to find drug and symptom names from the text.

Through this project, I learned a lot about building machine learning models, analyzing healthcare data, and creating systems that detect real-world risks.

TEACHER'S SIGNATURE

STUDENT'S SIGNATURE

SANSKRITI AGARWAL(2205846)

Abstract:

The project is about creating an intelligent healthcare system that helps doctors make better medical decisions using AI. It includes several parts like predicting diseases, suggesting the right medicines, and detecting side effects of drugs. Together, these parts form a full healthcare support tool.

My role was to build the medicine suggestion part.
I designed a system that recommends similar medicines by looking at their descriptions and how they are used.
I worked with a dataset containing more than 9,000 medicines.
I used text processing methods like breaking down words, shortening them, and turning them into numbers to create a way to compare medicines. Then, I used a method called cosine similarity to find out how similar medicines are and suggested the top ones.

I also improved the way the recommendations were made to make sure they were accurate and helpful.
I tested the system and checked if the suggestions were good.

This work helped me learn about recommendation systems, natural language processing, and models that use similarity to work in real-life healthcare settings.

TEACHER'S SIGNATURE

STUDENT'S SIGNATURE

RAJDEEP GANGULY(22053337)

Abstract:

This project uses machine learning to work with healthcare data, allowing for automatic predictions and smart suggestions that help improve patient care.

Individual Contribution and Findings:

I was in charge of Exploratory Data Analysis (EDA) and data preparation, which were the starting points for the whole project.
I looked at several datasets to learn about their structure, how the data was spread out, and important trends.

I did tasks like fixing missing information, removing repeated entries, and organizing text data.
I also made changes to the data so it could be used to train models, which included transforming the data and creating new features.

I made charts and summaries to better understand the data and help build the models.
My work made sure the data was clear, dependable, and ready for use with machine learning.

Throughout this process, I improved my skills in analyzing data, preparing data for machine learning, and handling real-world datasets for AI models.

TEACHER'S SIGNATURE	**STUDENT'S SIGNATURE**

SOURAMAY BHOWMIK(22052160)

Abstract:

The project is a full AI-based healthcare system that focuses on being easy to use and well connected with other systems. It lets users interact with several smart parts of the system through one simple interface.

Individual Contribution and Findings:

I worked on both the front and back parts of the system.
For the back end, I used FastAPI to connect all the machine learning models into one system that works through APIs. I made specific parts of the system for predicting diseases, suggesting medicines, and detecting adverse drug reactions. I made sure data moved smoothly between the user interface and the back end. I also took care of loading the models, handling user requests, and formatting the responses.

On the front end, I designed an easy-to-use interface that lets users upload files, enter data, and see results clearly.
I made sure the front and back ends worked together smoothly using API calls.

By doing this work, I learned a lot about building full-stack applications, designing APIs, and putting AI models into real-world healthcare systems.

TEACHER'S SIGNATURE

STUDENT'S SIGNATURE

SUMIT KUMAR MANDAL(22052166)

Abstract:

This project is about using deep learning methods to study clinical text data and accurately predict diseases. Using transformer-based models helps the system understand complex medical language better.

Individual Contribution and Findings:

I was in charge of adjusting the BERT model to predict diseases based on clinical notes.
My job included preparing the text data by cleaning it, making it standard, and removing unnecessary parts.

I used the HuggingFace Transformers library to set up the BERT model and trained it on a dataset that included clinical notes along with their related diagnoses.
I worked on adjusting settings like learning rate, batch size, and number of training rounds to make the model perform better.

I also checked the model's performance using measures like accuracy, precision, recall, and F1-score.
I also took care of saving the model and making it ready for use in the backend system.

Through this work, I learned a lot about deep learning, natural language processing, and how transformer models can be used in healthcare.

TEACHER'S SIGNATURE

STUDENT'S SIGNATURE

SOUMYA DHANKAR(22051465)

Abstract:

The project's goal is to create a full AI healthcare system by combining different data sets and smart models to give correct and useful results.

My role was to gather and organize the data needed for the project.
I found healthcare data sets related to disease prediction, medicine suggestions, and drug side effect detection from trusted sources.

I made sure the data was well-organized and matched the system's needs.
I also helped clean and prepare the data so it could be used to train the models.

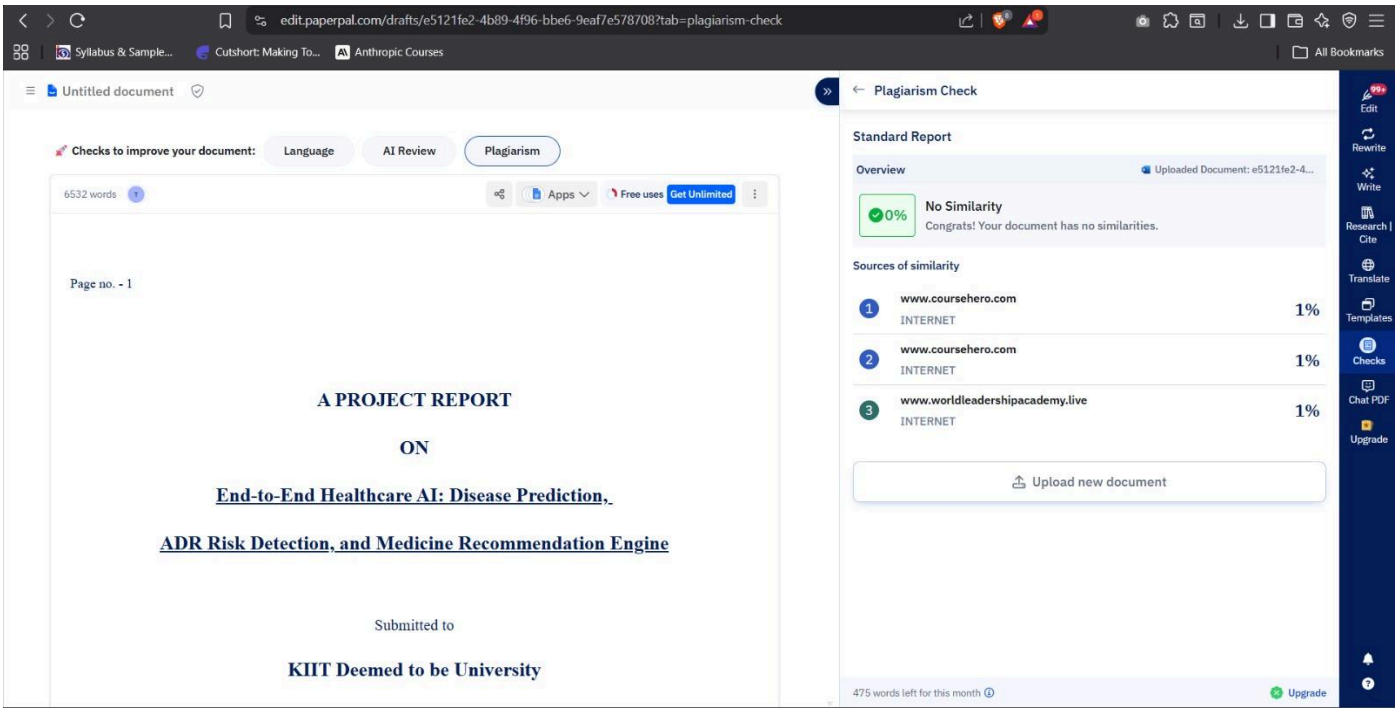
Alongside this, I worked on writing reports and keeping records, making sure the project was explained clearly and neatly.
I helped put together the results, format the sections, and keep everything in line throughout the report.

By doing this work, I learned about collecting data, writing reports, and the importance of structured data in machine learning projects.

TEACHER'S SIGNATURE

STUDENT'S SIGNATURE

PLAGARISM REPORT



CONCLUSION

This project successfully presents the design and implementation of an end-to-end AI-powered healthcare system that integrates three critical functionalities—disease prediction, medicine recommendation, and adverse drug reaction (ADR) detection—into a single unified platform. The system demonstrates how modern machine learning and natural language processing techniques can be effectively applied to solve real-world healthcare problems and assist in clinical decision-making.

The disease prediction module, built using a fine-tuned BERT model, showcases the ability of deep learning architectures to understand complex clinical text and accurately classify diseases based on patient notes. The contextual understanding provided by transformer-based models significantly improves prediction accuracy compared to traditional methods. The medicine recommendation system complements this by providing relevant and similar medicines using content-based filtering and cosine similarity, offering practical alternatives when needed. Additionally, the ADR detection module enhances patient safety by identifying potential adverse reactions through a hybrid approach that combines Named Entity Recognition, machine learning classification, and validation using a trusted medical database.

The implementation of the system using a FastAPI-based backend and a modern frontend framework ensures smooth interaction between components and real-time processing of user inputs. The modular architecture allows each component to function independently while still being part of a cohesive system, making it scalable and adaptable for future enhancements. The system is capable of handling multiple tasks efficiently and delivering results in a user-friendly manner, demonstrating its practicality in real-world scenarios.

Furthermore, the project highlights the importance of integrating multiple AI techniques into a single pipeline to address different stages of the healthcare workflow—from diagnosis to treatment recommendation and safety monitoring. The use of structured and unstructured datasets, along with proper preprocessing and evaluation techniques, ensures the reliability and effectiveness of the system.

However, despite its strong performance, the system has certain limitations. The models are trained on limited datasets, and their performance may vary when exposed to more diverse and noisy real-world medical data. The medicine recommendation system relies on textual similarity rather than deep pharmacological relationships, and the ADR detection module is limited by the coverage of available databases. Addressing these limitations through larger datasets, improved feature integration, and clinical validation will be essential for future deployment.

In conclusion, this project demonstrates the transformative potential of artificial intelligence in healthcare by providing a comprehensive, efficient, and intelligent solution for medical data analysis. It serves as a strong foundation for further research and development in AI-driven healthcare systems, with the potential to improve diagnostic accuracy, enhance treatment decisions, and ensure better patient safety.

REFERENCES

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019).
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
Proceedings of NAACL-HLT.
HuggingFace.
Transformers Library Documentation.
Available: https://huggingface.co/docs/transformers
FastAPI Documentation.
Available: https://fastapi.tiangolo.com/
Pedregosa, F., et al. (2011).
Scikit-learn: Machine Learning in Python.
Journal of Machine Learning Research.
spaCy Documentation.
Available: https://spacy.io/
NLTK Documentation.
Available: https://www.nltk.org/
Wishart, D. S., et al. (2018).
DrugBank: a comprehensive resource for drug information.
Nucleic Acids Research.
Kuhn, M., & Johnson, K. (2013).
Applied Predictive Modeling. Springer.
SIDER 4.1.
Available: http://sideeffects.embl.de/
Python Software Foundation.
Python Documentation.
Available: https://docs.python.org/